

# Predicting the Stock Market with Python

In this assignment, We will predict the Close/Last price of Microsoft stock using a Long Short-Term Memory neural network (LSTM) model. LSTM is a type of Recurrent Neural Network (RNN) that is capable of capturing and understand dependencies across multiple time steps or positions in the input sequence. This makes LSTMs particularly useful in various applications, including machine translation, sentiment analysis, speech recognition, and time series forecasting, where understanding and modeling the temporal relationships is crucial for accurate predictions.

We will use the sequence to sequence learning to make a multistep forecast. We will add two layers, a repeat vector layer and time distributed dense layer in the architecture. A repeat vector layer is used to repeat the context vector we get from the encoder to pass it as an input to the decoder. We will repeat it for n-steps ( n is the no of future steps you want to forecast). The output received from the decoder with respect to each time step is mixed. The time distributed densely will apply a fully connected dense layer on each time step and separates the output for each timestep. The time distributed densely is a wrapper that allows applying a layer to every temporal slice of an input.

We will stack additional layers on the encoder part and the decoder part of the sequence to sequence model. By stacking LSTM's, it may increase the ability of our model to understand more complex representation of our time-series data in hidden layers, by capturing information at different levels.

## Importing Libraries

```
import math
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
import os

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout
from tensorflow.keras.layers import RepeatVector, TimeDistributed

pd.options.mode.chained_assignment = None
import seaborn as sns
from matplotlib.pyplot import rcParams
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
```

## Loading Dataset

The dataset is 10 y.o. historical prices of Microsoft stock listed in the Nasdaq market. From the data, we select the values of the Close/Last column as the dataset for the model. The "Open" column represents the opening price for shares that day and the "High" column represents the highest price shares reached that day. "Low" represents the lowest share price for the day, "Last" represents the price at which the last transaction for a share went through. "Close" represents the price shares ended at for the day.

```

# Loading the dataset
msft_df =pd.read_csv('MSFT_stock.csv',parse_dates=['Date'],
                    index_col=['Date'], low_memory=False)
msft_df.rename(columns={'Close/Last': 'Close'},
               inplace=True, errors='raise')
msft_df.sort_values(by=['Date'], inplace=True)

# Remove currency symbols and convert dtype to float.
msft_df['Close'] = msft_df['Close'].str.replace('$', '',
                                                regex=False).astype(float)
msft_df['Open'] = msft_df['Open'].str.replace('$', '',
                                                regex=False).astype(float)
msft_df['High'] = msft_df['High'].str.replace('$', '',
                                                regex=False).astype(float)
msft_df['Low'] = msft_df['Low'].str.replace('$', '',
                                              regex=False).astype(float)
msft_df.head()

```

|            | Close | Volume   | Open  | High  | Low    |
|------------|-------|----------|-------|-------|--------|
| Date       |       |          |       |       |        |
| 2013-07-29 | 31.54 | 28870360 | 31.47 | 31.60 | 31.400 |
| 2013-07-30 | 31.85 | 45799480 | 31.78 | 32.12 | 31.550 |
| 2013-07-31 | 31.84 | 43898440 | 31.97 | 32.05 | 31.710 |
| 2013-08-01 | 31.67 | 42499400 | 32.06 | 32.09 | 31.600 |
| 2013-08-02 | 31.89 | 29120110 | 31.69 | 31.90 | 31.565 |

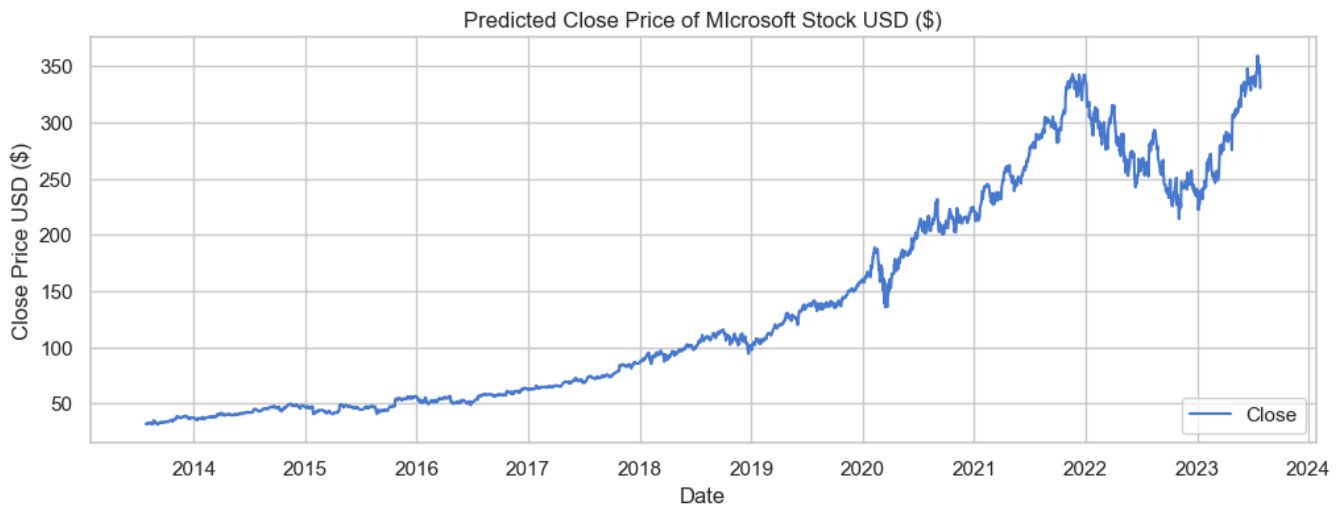
## Visualizaing Data

It is important to show the historical close prices movement of Microsoft (MSFT) by plotting a line chart.

```

# Visualizing Close prices of Microsoft
plt.figure(figsize=(12, 4))
plt.title('Predicted Close Price of MIncrosoft Stock USD ($)')
plt.xlabel('Date')
plt.ylabel('Close Price USD ($)')
plt.plot(msft_df.Close)
plt.legend(['Close'], loc='lower right')
plt.show()

```



## Data Preporcessing

### 1. Preparation of Training and Test Dataset

The number of instances is 2,517. We will split the Close prices into train and test data in a 75% and 25% ratio of the instances (ie., 1,887 training dataset and 630 test dataset).

```
# Split Close prices into training data and test data.
msft = msft_df.iloc[:, :1].values

train_size = int(len(msft) * 0.75)
test_size = len(msft) - train_size

train_size, test_size
```

(1887, 630)

### 2. Data Normalization

Normalization is changing the values of numeric columns in the dataset to a common scale, which helps the performance of our model. To scale the dataset we use Scikit-Learn's MinMaxScaler with numbers between zero and one.

```
# Scaling dataset with numbers between 0 and 1
sc = MinMaxScaler(feature_range=(0,1))
sc_msft = sc.fit_transform(msft.reshape(-1,1))

train_msft = sc_msft[0: train_size, :]
test_msft = sc_msft[train_size-30: , : ]
```

### 3. Incorporating Timesteps Into Data

Before we use this data to train our model, we need to split the data into subsequences because this is a time series modelling project. We can create specific sequences with specific time steps. In this case, we will use time steps as 30. We can create sequences worth 30 days of historical data and as required by LSTM, we need to reshape our input data into n times the shape and sample features.

```
# Create 30 days of historical data and reshape it.
time_steps = 30

X_train = []
y_train = []

for i in range(time_steps, len(train_msft)):
    X_train.append(train_msft_sc[i-time_steps:i, 0])
    y_train.append(train_msft_sc[i, 0])
X_train, y_train = np.array(X_train), np.array(y_train)

X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
```

```
x_test = []
y_test = msft[train_size:]

for i in range(time_steps, len(test_msft)):
    x_test.append(test_msft[i-time_steps:i, 0])

x_test = np.array(x_test)
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))
```

## Model Architecture

We will be using the Sequential model class from Keras API. A LSTM layer will be added with the following arguments: 128 units (this is the dimensionality of the output space), set `return_sequences=True` (this is necessary for stacking LSTM layers so the consequent LSTM layer has a three-dimensional sequence input), and `input_shape` (this is the shape of the training dataset). We will be specifying 0.2 in the Dropout layer, meaning that 20% of the layers will be dropped. Since the subsequent layer is an LSTM, we need to duplicate this using a RepeatVector layer. This RepeatVector layer repeats the feature vector layer from the output of the LSTM timesteps times which is 30 in this case. Now to create the decoder, we mirror the encoder in reverse fashion to create our decoder. Then we add Time distributed layer so that we can get the output of the right shape where the number of nodes is equal to number of features. The time distributed layer creates a vector equal to the number of layers output from the previous layer. We add another LSTM layer with 64 network units. But we set the `return_sequence` to false for this time to only return the last output in the output sequence. Lastly, we add a densely connected layer that specifies the output of 1 network unit. Mean squared error is the loss function for optimizing the model with adam optimizer.

```
timesteps, num_features = X_train.shape[1], X_train.shape[2]
```

```

model = Sequential([
    LSTM(128, input_shape=(timesteps, num_features)),
    RepeatVector(timesteps),
    LSTM(128, return_sequences=True),
    TimeDistributed(Dense(64)),
    LSTM(units=64),
    Dense(num_features)
], name='Stock_Predictor')

model.compile(loss='mean_squared_error', optimizer='adam')
model.summary()

```

Model: "Stock\_Predictor"

| Layer (type)                          | Output Shape    | Param # |
|---------------------------------------|-----------------|---------|
| lstm_41 (LSTM)                        | (None, 128)     | 66560   |
| repeat_vector_12 (RepeatVector)       | (None, 30, 128) | 0       |
| lstm_42 (LSTM)                        | (None, 30, 128) | 131584  |
| time_distributed_12 (TimeDistributed) | (None, 30, 64)  | 8256    |
| lstm_43 (LSTM)                        | (None, 64)      | 33024   |
| dense_24 (Dense)                      | (None, 1)       | 65      |
| Total params: 239489 (935.50 KB)      |                 |         |
| Trainable params: 239489 (935.50 KB)  |                 |         |
| Non-trainable params: 0 (0.00 Byte)   |                 |         |

From the above model configuration, we can see that the Repeat Vector just duplicated the encoded representation 30 times. The time distributed layer decode the output layer and created 8256. Now let's train the model.

## Train the model

```

model.fit(X_train,y_train,epochs=100,batch_size=32,shuffle=False)

```

Epoch 1/100  
59/59 [=====] - 12s 55ms/step - loss: 0.0315  
Epoch 2/100  
59/59 [=====] - 6s 95ms/step - loss: 0.0303  
Epoch 3/100  
59/59 [=====] - 5s 92ms/step - loss: 0.0298  
Epoch 4/100  
59/59 [=====] - 2s 42ms/step - loss: 0.0167  
Epoch 5/100  
59/59 [=====] - 3s 46ms/step - loss: 0.0165  
Epoch 6/100  
59/59 [=====] - 5s 88ms/step - loss: 0.0109  
Epoch 7/100  
59/59 [=====] - 5s 83ms/step - loss: 0.0101  
Epoch 8/100  
59/59 [=====] - 3s 51ms/step - loss: 0.0088  
Epoch 9/100  
59/59 [=====] - 3s 47ms/step - loss: 0.0079  
Epoch 10/100  
59/59 [=====] - 5s 83ms/step - loss: 0.0062  
Epoch 11/100  
59/59 [=====] - 6s 98ms/step - loss: 0.0045  
Epoch 12/100  
59/59 [=====] - 3s 50ms/step - loss: 0.0031  
Epoch 13/100  
59/59 [=====] - 3s 49ms/step - loss: 0.0019  
Epoch 14/100  
59/59 [=====] - 5s 85ms/step - loss: 0.0013  
Epoch 15/100  
59/59 [=====] - 5s 93ms/step - loss: 0.0011  
Epoch 16/100  
59/59 [=====] - 3s 53ms/step - loss: 9.3514e-04  
Epoch 17/100  
59/59 [=====] - 3s 51ms/step - loss: 8.0358e-04  
Epoch 18/100  
59/59 [=====] - 5s 91ms/step - loss: 7.0627e-04  
Epoch 19/100  
59/59 [=====] - 5s 84ms/step - loss: 6.1492e-04  
Epoch 20/100  
59/59 [=====] - 3s 49ms/step - loss: 5.3107e-04  
Epoch 21/100  
59/59 [=====] - 3s 53ms/step - loss: 4.4842e-04  
Epoch 22/100  
59/59 [=====] - 5s 92ms/step - loss: 3.6941e-04  
Epoch 23/100  
59/59 [=====] - 5s 78ms/step - loss: 2.9686e-04  
Epoch 24/100  
59/59 [=====] - 3s 48ms/step - loss: 2.3777e-04  
Epoch 25/100  
59/59 [=====] - 3s 48ms/step - loss: 1.9777e-04  
Epoch 26/100  
59/59 [=====] - 6s 97ms/step - loss: 1.7797e-04  
Epoch 27/100  
59/59 [=====] - 5s 80ms/step - loss: 1.7376e-04  
Epoch 28/100  
59/59 [=====] - 3s 48ms/step - loss: 1.7889e-04  
Epoch 29/100  
59/59 [=====] - 3s 50ms/step - loss: 1.8781e-04  
Epoch 30/100  
59/59 [=====] - 6s 94ms/step - loss: 1.9872e-04  
Epoch 31/100  
59/59 [=====] - 5s 83ms/step - loss: 2.1344e-04  
Epoch 32/100  
59/59 [=====] - 3s 47ms/step - loss: 2.3320e-04

Epoch 33/100  
59/59 [=====] - 3s 56ms/step - loss: 2.5862e-04  
Epoch 34/100  
59/59 [=====] - 5s 92ms/step - loss: 2.9180e-04  
Epoch 35/100  
59/59 [=====] - 4s 72ms/step - loss: 3.3639e-04  
Epoch 36/100  
59/59 [=====] - 3s 49ms/step - loss: 3.9943e-04  
Epoch 37/100  
59/59 [=====] - 3s 55ms/step - loss: 4.9586e-04  
Epoch 38/100  
59/59 [=====] - 6s 96ms/step - loss: 6.5886e-04  
Epoch 39/100  
59/59 [=====] - 4s 75ms/step - loss: 9.6751e-04  
Epoch 40/100  
59/59 [=====] - 3s 52ms/step - loss: 0.0016  
Epoch 41/100  
59/59 [=====] - 3s 50ms/step - loss: 0.0031  
Epoch 42/100  
59/59 [=====] - 5s 92ms/step - loss: 0.0059  
Epoch 43/100  
59/59 [=====] - 4s 76ms/step - loss: 0.0096  
Epoch 44/100  
59/59 [=====] - 3s 49ms/step - loss: 0.0103  
Epoch 45/100  
59/59 [=====] - 4s 60ms/step - loss: 0.0075  
Epoch 46/100  
59/59 [=====] - 6s 97ms/step - loss: 0.0097  
Epoch 47/100  
59/59 [=====] - 4s 69ms/step - loss: 0.0067  
Epoch 48/100  
59/59 [=====] - 3s 49ms/step - loss: 0.0037  
Epoch 49/100  
59/59 [=====] - 4s 64ms/step - loss: 0.0014  
Epoch 50/100  
59/59 [=====] - 6s 95ms/step - loss: 3.1403e-04  
Epoch 51/100  
59/59 [=====] - 4s 66ms/step - loss: 8.7608e-04  
Epoch 52/100  
59/59 [=====] - 3s 49ms/step - loss: 1.6669e-04  
Epoch 53/100  
59/59 [=====] - 4s 68ms/step - loss: 0.0010  
Epoch 54/100  
59/59 [=====] - 5s 91ms/step - loss: 2.3388e-04  
Epoch 55/100  
59/59 [=====] - 4s 65ms/step - loss: 0.0016  
Epoch 56/100  
59/59 [=====] - 3s 47ms/step - loss: 2.9062e-04  
Epoch 57/100  
59/59 [=====] - 4s 71ms/step - loss: 0.0020  
Epoch 58/100  
59/59 [=====] - 6s 98ms/step - loss: 3.0091e-04  
Epoch 59/100  
59/59 [=====] - 4s 61ms/step - loss: 0.0020  
Epoch 60/100  
59/59 [=====] - 3s 45ms/step - loss: 3.1631e-04  
Epoch 61/100  
59/59 [=====] - 5s 79ms/step - loss: 0.0020  
Epoch 62/100  
59/59 [=====] - 6s 96ms/step - loss: 3.0171e-04  
Epoch 63/100  
59/59 [=====] - 3s 53ms/step - loss: 0.0018  
Epoch 64/100  
59/59 [=====] - 3s 49ms/step - loss: 2.9894e-04

Epoch 65/100  
59/59 [=====] - 5s 81ms/step - loss: 0.0017  
Epoch 66/100  
59/59 [=====] - 5s 88ms/step - loss: 2.6937e-04  
Epoch 67/100  
59/59 [=====] - 3s 50ms/step - loss: 0.0014  
Epoch 68/100  
59/59 [=====] - 3s 53ms/step - loss: 2.5457e-04  
Epoch 69/100  
59/59 [=====] - 4s 75ms/step - loss: 0.0013  
Epoch 70/100  
59/59 [=====] - 6s 95ms/step - loss: 2.2239e-04  
Epoch 71/100  
59/59 [=====] - 3s 49ms/step - loss: 0.0010  
Epoch 72/100  
59/59 [=====] - 3s 54ms/step - loss: 2.0308e-04  
Epoch 73/100  
59/59 [=====] - 5s 79ms/step - loss: 8.4534e-04  
Epoch 74/100  
59/59 [=====] - 6s 95ms/step - loss: 1.7627e-04  
Epoch 75/100  
59/59 [=====] - 3s 48ms/step - loss: 6.2749e-04  
Epoch 76/100  
59/59 [=====] - 3s 50ms/step - loss: 1.5884e-04  
Epoch 77/100  
59/59 [=====] - 5s 85ms/step - loss: 4.8214e-04  
Epoch 78/100  
59/59 [=====] - 5s 88ms/step - loss: 1.3903e-04  
Epoch 79/100  
59/59 [=====] - 3s 47ms/step - loss: 3.4666e-04  
Epoch 80/100  
59/59 [=====] - 3s 53ms/step - loss: 1.2425e-04  
Epoch 81/100  
59/59 [=====] - 4s 75ms/step - loss: 2.4867e-04  
Epoch 82/100  
59/59 [=====] - 5s 93ms/step - loss: 1.1938e-04  
Epoch 83/100  
59/59 [=====] - 3s 49ms/step - loss: 1.7766e-04  
Epoch 84/100  
59/59 [=====] - 3s 54ms/step - loss: 1.3876e-04  
Epoch 85/100  
59/59 [=====] - 5s 85ms/step - loss: 1.5210e-04  
Epoch 86/100  
59/59 [=====] - 5s 92ms/step - loss: 1.5986e-04  
Epoch 87/100  
59/59 [=====] - 3s 45ms/step - loss: 1.6577e-04  
Epoch 88/100  
59/59 [=====] - 3s 52ms/step - loss: 1.7540e-04  
Epoch 89/100  
59/59 [=====] - 5s 86ms/step - loss: 1.8963e-04  
Epoch 90/100  
59/59 [=====] - 5s 89ms/step - loss: 2.0181e-04  
Epoch 91/100  
59/59 [=====] - 3s 44ms/step - loss: 2.0796e-04  
Epoch 92/100  
59/59 [=====] - 3s 53ms/step - loss: 2.0971e-04  
Epoch 93/100  
59/59 [=====] - 5s 86ms/step - loss: 2.0680e-04  
Epoch 94/100  
59/59 [=====] - 5s 85ms/step - loss: 1.9874e-04  
Epoch 95/100  
59/59 [=====] - 3s 48ms/step - loss: 1.8698e-04  
Epoch 96/100  
59/59 [=====] - 3s 58ms/step - loss: 1.7320e-04



```
Epoch 97/100
59/59 [=====] - 5s 79ms/step - loss: 1.5918e-04
Epoch 98/100
59/59 [=====] - 5s 87ms/step - loss: 1.4672e-04
Epoch 99/100
59/59 [=====] - 3s 48ms/step - loss: 1.3766e-04
Epoch 100/100
59/59 [=====] - 3s 53ms/step - loss: 1.3407e-04
<keras.src.callbacks.History at 0x1aed08a41c0>
```

## Making Predictions on the future stock (Test Set)

We apply the model to predict the stock prices based on the test set. We use the `inverse_transform` method to denormalize the predicted stock prices. We apply the RMSE formula to calculate the degree of discrepancy between the predicted prices and real prices ( $y_{test}$ ) and display the result.

```
predictions = model.predict(x_test)
predictions = sc.inverse_transform(predictions)
rmse = np.sqrt(np.mean(predictions - y_test)**2)
rmse
```

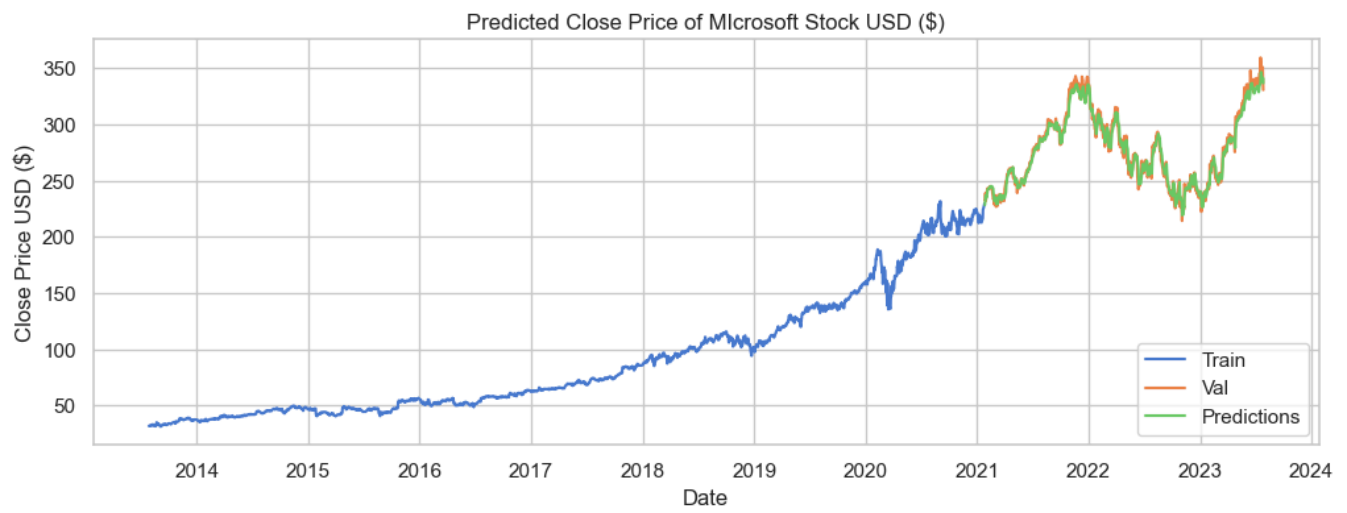
```
20/20 [=====] - 2s 35ms/step
1.1528431987459704
```

The result shows that the RMSE is as low as about **1.15**. This means the model is working well.

## Plotting the Results

At this stage, we use matplotlib to visualize the result of our predicted stock price and the actual stock price. We use the filter method to only retain the closing price column in the dataframe. We then split our stock data into three plotting regions: training, validation and prediction. Finally, we configure the chart figure size, title, x-axis & y-axis label and legends.

```
data = msft_df.filter(['Close'])
train = data[:train_size]
validation = data[train_size:]
validation['Predictions'] = predictions
plt.figure(figsize=(12, 4))
plt.title('Predicted Close Price of MICROsoft Stock USD ($)')
plt.xlabel('Date')
plt.ylabel('Close Price USD ($)')
plt.plot(train)
plt.plot(validation[['Close', 'Predictions']])
plt.legend(['Train', 'Val', 'Predictions'], loc='lower right')
plt.show()
```



From the resulting chart above, we can see the predicted stock prices follow the trend of the real stock prices closely. This shows the effectiveness of the LSTM to work with time series or sequential data like the stock prices.